Desafío final: captura, de-codificación y recuperación de datos en redes de computadoras

Objetivos:

- Comprender el principio de los códigos de detección y corrección de errores (EDAC).
- Aplicar a mano el código de paridad de filas y columnas (paridad bidimensional) para detectar, ubicar y corregir errores.
- Capturar con Wireshark un mensaje de difusión (broadcast) en una red local e interpretar su contenido.
- Reconstruir la información original a partir de un mensaje posiblemente dañado.

Requerimientos:

- Al menos una computadora con Wi-Fi por grupo (o un teléfono + computadora).
- Wireshark (o cualquier método de captura que quieran)
- Python 3, o una cuenta de Google para usar Google Colab. Consideren que una compu va a estar conectada a la red del aula, por lo que no va a tener internet (necesario para Colab).
- Scripts decodificar_mensaje.py, paridad.py, o sus versiones .ipyn

La pregunta que guía todo el TP:

`Un mensaje viaja por un canal que no es perfecto y llega con algunos bits cambiados. ¿Cómo puede el receptor darse cuenta, y hasta dónde puede arreglarlo sin pedir que se lo reenvíen?`

Durante la clase, una computadora de la cátedra va a estar transmitiendo un mensaje dentro de la red Wi-Fi del laboratorio. El mensaje está protegido con el código que se describe más adelante y es posible que no llegue intacto. El objetivo es capturarlo, verificarlo, corregirlo si hace falta y reconstruir lo que contiene.

Antes de empezar el desafío deberán:

- Conectarse a la red Wi-Fi que indique la cátedra. Si no están en esa red, no van a ver nada.
- Capturar solamente en esa red y en sus propias computadoras.
- En Windows, usar python (o py); en Linux/macOS, python3. En los ejemplos escribimos python.
- Deberán guardar la captura donde aparezca el mensaje (Archivo → Guardar como..., formato .pcapng, o captura de pantalla). Esto forma parte de la entrega.

Marco teórico: repasar y tener muy en claro antes del encuentro

Repasar bits de paridad:

La forma más simple de detectar errores es agregar a un bloque de datos un bit de paridad. Con paridad par, el bit agregado se elige de forma que la cantidad total de unos (datos + paridad) sea par. Dicho de otra forma, el bit de paridad es el XOR de todos los bits de datos:

1011 tiene tres unos (impar) → bit de paridad 1.
0101 tiene dos unos (par) → bit de paridad 0.

El receptor recalcula la paridad de lo que recibió y la compara con el bit recibido. Si no coinciden, hubo un error. Pero con un único bit de paridad:

- ➔ un error en un bit se detecta, pero no se sabe cuál bit cambió, así que no se puede corregir.
- ➔ un error en dos bits no se detecta (la paridad vuelve a cerrar).

Entender paridad columna-fila o código producto de paridad:

La idea es organizar los bits de datos como una matriz de N filas por M columnas y calcular un bit de paridad (par) por cada fila y otro por cada columna. A este esquema también se lo llama código producto de paridad.

Ejemplo con un bloque de 16 bits organizado en 4 × 4:

```
                paridad
datos           de fila
1 0 1 1   →      1       (tres unos)
0 1 0 1   →      0
1 1 1 1   →      0
0 1 0 1   →      0
↓ ↓ ↓ ↓
0 1 0 0      ← paridad de columna
```

En este laboratorio los bits de control se transmiten primero los de las filas (de arriba hacia abajo) y luego los de las columnas (de izquierda a derecha). En el ejemplo, los 8 bits de control son:

➔ 1000 0100

Detección y corrección

El receptor reorganiza los bits recibidos en la matriz, calcula la paridad de cada fila y de cada columna y la compara con los bits de control recibidos. Las filas y columnas que no cierran se llaman **síndrome del error.**

Si durante la transmisión se invirtió un solo bit de datos, fallan exactamente una fila y una columna: el bit erróneo está en su intersección. Como en binario un bit sólo puede tener dos valores, corregir es simplemente invertirlo.

Ejemplo: supongamos que se recibe la matriz anterior con un bit cambiado (marcado entre corchetes) y los mismos bits de control 1000 0100:

```
                    paridad de filas
                    calculada                recibida
1   0   1   1       1                        1   ✔
0   1   0   1       0                        0   ✔
1   1  [0]  1       1                        0   ✘    ← fila 3
0   1   0   1       0                        0   ✔

0   1   1   0   ← Paridad de columnas calculada
0   1   0   0   ← Paridad de columnas recibida
✔   ✔   ✘   ✔

        ← columna 3
```

El error está en la fila 3, columna 3. Si invertimos ese bit, la matriz vuelve a ser consistente.

Prepara el entorno:

Antes de ir a buscar el mensaje, verifiquen que pueden traducir un archivo de unos y ceros a texto. El script decodificar_mensaje.py (convenientemente incluido dentro del drive de la materia) toma los bits de a 7 y los interpreta como caracteres ASCII (por ejemplo, 1000001 → A). Cada línea del archivo se decodifica por separado.

Con el archivo ejemplo provisto:

```
python decodificar_mensaje.py mensaje.txt
```

Debería mostrar un texto en pantalla. El script paridad.py muestra la matriz de bits con la paridad par de cada fila (a la derecha) y de cada columna (abajo):

```
python paridad.py mensaje.txt
```

Por ejemplo, para un archivo con las filas 1001, 0101 y 1001:

```
1001 |0
0101 |0
1001 |0
------
0101
```

Si tienen problemas con Python, pueden usar decodificar_mensaje.ipynb y paridad.ipynb en Google Colab. Peguen las filas de unos y ceros en la celda y ejecútenla.

El Desafío / Consignas

Conéctense a la red del laboratorio, utilizando DHCP. La computadora transmisora envía el mensaje a todos los equipos de la red (difusión) aproximadamente cada 30 segundos. No sabemos de antemano qué protocolo de transporte, qué puerto ni qué IP de origen usa: descubrirlo es parte del TP. Si los podemos ayudar diciéndoles que la carga útil tiene un formato de tipo JSON:

```
{"SIZE": "10,5", ???????????????? }
```

Dónde SIZE indica el tamaño de la matriz en formato FILAS, COLUMNAS

Atención: varias cosas las pueden responder luego del desafío, prioricen decodificar el mensaje durante el encuentro sincrónico!

1) Averigüen su IP y su máscara (ipconfig / ip addr) y calculen la dirección de broadcast de la red. ¿Cuál es la dirección MAC de broadcast?

2) Abran Wireshark en la interfaz Wi-Fi y capturen al menos 2 minutos. Pueden llegar varios mensajes; no tienen porqué ser todos idénticos.

3) Analicen la estructura del mensaje. ¿Qué formato tiene? ¿Qué campos contiene y qué creen que representa cada uno? Con lo visto en la sección 1, propongan cómo hay que interpretar los datos.

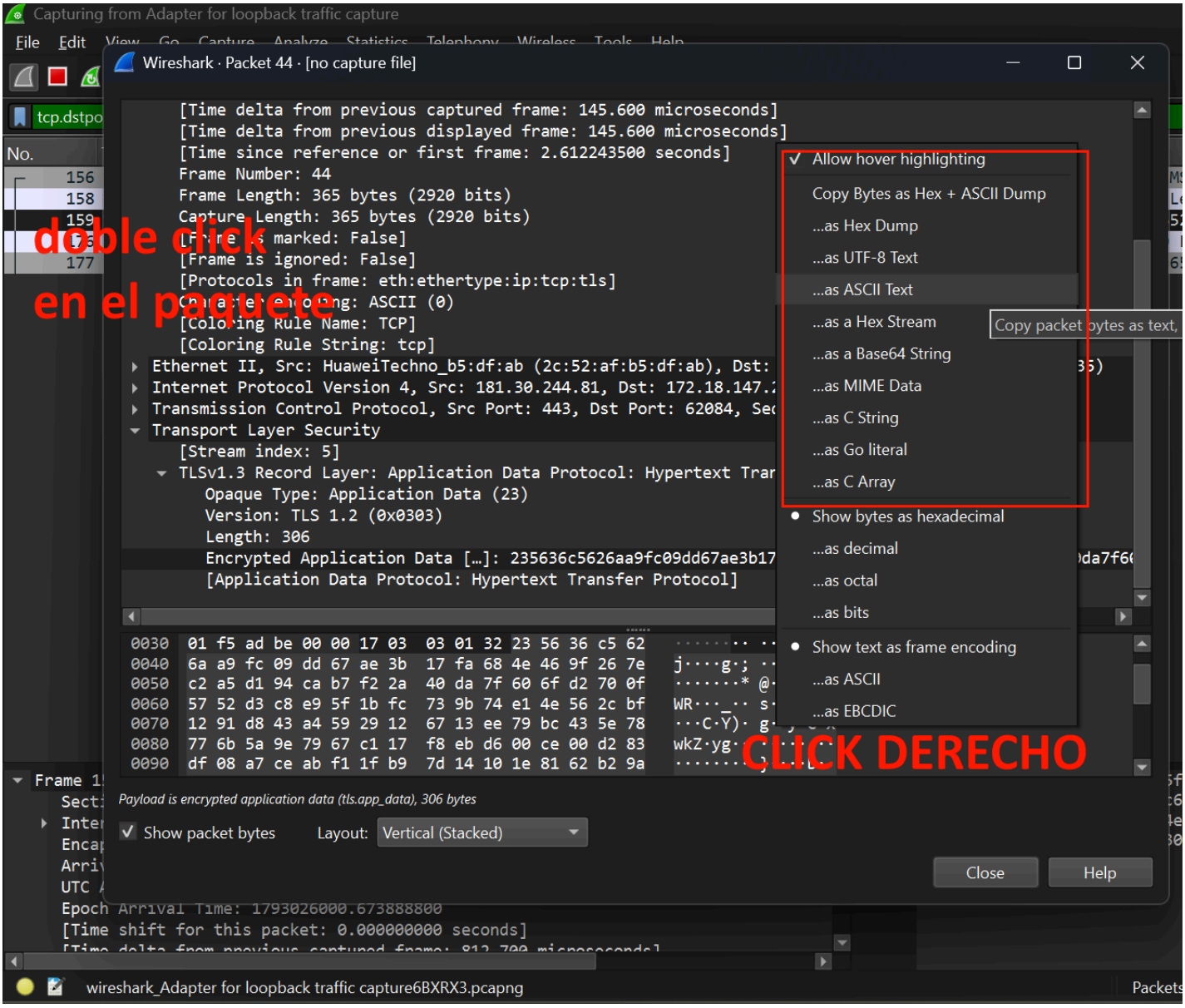
4) Verifiquen la integridad del mensaje con el código de la sección 1: recalculen las paridades y compárenlas con las recibidas. Registren qué filas y qué columnas fallan (el síndrome).

5) Si hay errores, ubíquenlos y corríjanlos. Para cada bit corregido indiquen fila, columna, valor recibido y valor corregido. Si recibieron más de un mensaje distinto, analicen cada uno por separado.

6) Construyan un archivo de texto en el formato de mensaje.txt con los datos ya corregidos y decodifíquenlo con decodificar_mensaje.py (o el cuaderno). Si todo salió bien, el mensaje les va a indicar cómo seguir. Ojo: un mensaje que "se lee bien" no es necesariamente un mensaje correcto.

7) (Opcional) paridad.py calcula las paridades, pero no las compara con las recibidas.

Ayuda: en wireshark pueden copiar la carga útil de una captura como texto ASCII fácilmente:



Aclaración importante: si, las cargas útiles son 1 y 0 ASCII, texto, no literalmente bits. Entendamos: cada uno y cero ocupa 7 bits, su representación ASCII para el laboratorio es solamente por el valor didáctico de la actividad, en la vida real una técnica EDAC por supuesto utiliza una lógica bit-wise.

Reflexiones e investigación

1) ¿Qué hubiera pasado si el mensaje tenía errores en dos bits de filas y columnas distintas? ¿Y si se dañaba un bit de los de control?

2) Si el mensaje es texto en castellano, muchas veces un error "se adivina" por contexto (Hola mondo). ¿Sirve eso para una contraseña, un número de cuenta o una dirección web? ¿Qué aporta el código frente a "leer y adivinar"?

3) ¿Pudieron confirmar que su corrección es correcta? ¿Cómo?

4) Investigar y describir con el mayor detalle posible cómo funciona la técnica de corrección **Hamming (7,4)** y por qué es superior a la demostración didáctica que hicimos en este Laboratorio. Luego, aplicar la técnica sobre la carga útil obtenida en el desafío y elaborar un paquete de datos que podrían transmitir para que, en caso de corrupción, pueda ser recuperado.



ASCII	Char	Hex	Bin
32	space	20	0010 0000
33	!	21	0010 0001
34	"	22	0010 0010
35	#	23	0010 0011
36	$	24	0010 0100
37	%	25	0010 0101
38	&	26	0010 0110
39	'	27	0010 0111
40	(	28	0010 1000
41	)	29	0010 1001
42	*	2A	0010 1010
43	+	2B	0010 1011
44	,	2C	0010 1100
45	-	2D	0010 1101
46	.	2E	0010 1110
47	/	2F	0010 1111
48	0	30	0011 0000
49	1	31	0011 0001
50	2	32	0011 0010
51	3	33	0011 0011
52	4	34	0011 0100
53	5	35	0011 0101

ASCII	Char	Hex	Bin
54	6	36	0011 0110
55	7	37	0011 0111
56	8	38	0011 1000
57	9	39	0011 1001
58	:	3A	0011 1010
59	;	3B	0011 1011
60	<	3C	0011 1100
61	=	3D	0011 1101
62	>	3E	0011 1110
63	?	3F	0011 1111
64	@	40	0100 0000
91	[	5B	0101 1011
92	\	5C	0101 1100
93	]	5D	0101 1101
94	^	5E	0101 1110
95	_	5F	0101 1111
96	`	60	0110 0000
123	{	7B	0111 1011
124	|	7C	0111 1100
125	}	7D	0111 1101
126	~	7E	0111 1110

ASCII	Char	Hex	Bin	ASCII	Char	Hex	Bin
65	A	41	0100 0001	97	a	61	0110 0001
66	B	42	0100 0010	98	b	62	0110 0010
67	C	43	0100 0011	99	c	63	0110 0011
68	D	44	0100 0100	100	d	64	0110 0100
69	E	45	0100 0101	101	e	65	0110 0101
70	F	46	0100 0110	102	f	66	0110 0110
71	G	47	0100 0111	103	g	67	0110 0111
72	H	48	0100 1000	104	h	68	0110 1000
73	I	49	0100 1001	105	i	69	0110 1001
74	J	4A	0100 1010	106	j	6A	0110 1010
75	K	4B	0100 1011	107	k	6B	0110 1011
76	L	4C	0100 1100	108	l	6C	0110 1100
77	M	4D	0100 1101	109	m	6D	0110 1101
78	N	4E	0100 1110	110	n	6E	0110 1110
79	O	4F	0100 1111	111	o	6F	0110 1111
80	P	50	0101 0000	112	p	70	0111 0000
81	Q	51	0101 0001	113	q	71	0111 0001
82	R	52	0101 0010	114	r	72	0111 0010
83	S	53	0101 0011	115	s	73	0111 0011
84	T	54	0101 0100	116	t	74	0111 0100
85	U	55	0101 0101	117	u	75	0111 0101
86	V	56	0101 0110	118	v	76	0111 0110
87	W	57	0101 0111	119	w	77	0111 0111
88	X	58	0101 1000	120	x	78	0111 1000
89	Y	59	0101 1001	121	y	79	0111 1001
90	Z	5A	0101 1010	122	z	7A	0111 1010